Checking if a union alternative is active

Document #:	P2641R4
Date:	2023-06-14
Project:	Programming Language C++
Audience:	CWG, LWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com> Daveed Vandevoorde <daveed@edg.com>

Contents

1 Revision History
2 Introduction
3 Proposal
  3.1 Active Member of a Union, or Something More?
  3.2 Interesting Cases
    3.2.1 Nested Anonymous Unions
    3.2.2 Subobject
    3.2.3 Other rules
  3.3 Alternative Naming
  3.4 Why a pointer rather than a reference?
  3.5 Implementation Experience
4 Wording
  4.1 Feature-test Macro
5 References

§ 1 Revision History

Since [P2641R3], wording improvements.

Since [P2641R2], added a feature-test macro and a section explaining why this takes a pointer (not a reference).

After discussion in Issaquah, generalizing the proposed facility to check for object lifetime, instead of just active member of union.

§ 2 Introduction

Let's say you want to implement an `Optional<bool>` such that `sizeof(Optional<bool>) == 1`. This is conceptually doable since `bool` only has 2 states, but takes up a whole byte anyway, which leaves 254 bit patterns to use to express other criteria (i.e. that the `Optional` is disengaged).

There are two ways to do this, neither of which have undefined behavior:

Union	Reinterpret

`struct OptBool {`
`  union { bool b; char c; };`
 	
`  OptBool() : c(2) { }`
`  OptBool(bool b) : b(b) { }`
 	
`  auto has_value() const -> bool {`
`  return c != 2;`
`  }`
 	
`  auto operator*() -> bool& {`
`  return b;`
`  }`
`};`
	`struct OptBool {`
	`  char c;`
	
	`  OptBool() : c(2) { }`
	`  OptBool(bool b) { new (&c) bool(b); }`
	
	`  auto has_value() const -> bool {`
	`  return c != 2;`
	`  }`
	
	`  auto operator*() -> bool& {`
	`  return (bool&)c;`
	`  }`
	`};`

Both of these are fine: `operator*` works because we are returning a reference to a very-much-live `bool` in both cases, and `has_value()` works because we are allowed read through a `char` (explicitly, per [basic.lval]/11.3[1]).

However, try to make this work in `constexpr` and we immediately run into problems.

The union solution has a problem with `has_value()` because we're not allowed to read the non-active member of a union ([expr.const]/5.9). `c` isn't the active member of that union, so we can't read from it, even though it's `char`.

The reinterpet solution has two problems:

— the placement new into `&c` doesn't work. Placement-new is explicitly disallowed by [expr.const]/5.17. While we have `std::construct_at()` now to work around the lack of placement-new, that API is typed and would require a `bool*`, which we don't have.

— `operator*()` involves a `reinterpret_cast`, which is explicitly disallowed by [expr.const]/5.15.

This is unfortunate - we have a well-defined runtime solution that we cannot use during constant evaluation time. It'd be nice to do better.

§ 3 Proposal

Supporting the reinterpret solution is expensive - since it requires the implementation to track multiple types for the same bytes. But supporting the union solution, because it's more structured (and thus arguably better anyway), is very feasible. After all, our problem is localized to `has_value()`.

During runtime, we do need to read from `c`. But during constant evaluation time, we actually don't. What we need `c` for is to know whether `b` is the active member of the union or not. But the compiler already

knows whether `b` is the active member or not. Indeed, that is precisely *why* it's rejecting this implementation. What if we could simply ask the compiler this question?

```
struct OptBool {
  union { bool b; char c; };

  constexpr OptBool() : c(2) { }
  constexpr OptBool(bool b) : b(b) { }

  constexpr auto has_value() const -> bool {
    if consteval {
      return std::is_within_lifetime(&b);
    } else {
      return c != 2;
    }
  }

  constexpr auto operator*() -> bool& {
    return b;
  }
};
```

There's no particular implementation burden here - the compiler already needs to know this information. No language change is necessary to support this change either.

§ 3.1 Active Member of a Union, or Something More?

R0 and R1 of this paper [P2641R1] originally proposed a facility spelled `std::is_active_member(p)`, which asked if `p` was the active member of a union (or a subobject thereof). That is a sufficient question to solve the motivating example, but it's also a very narrow one. Can we generalize further?

To start with, asking if an object is the active member of a union is just a specific case of asking if an object is within its lifetime (as noted by Richard Smith). We see no need to be so specific with this facility, the more general question seems more useful.

But then the question becomes: do we go further? The most broad question would be to ask if evaluating an expression, any expression, as a constant would actually succeed. Perhaps such a (language) facility could be spelled as follows (mirroring the `requires` expression that we already have):

```
if (constexpr { e; }) {
    // evaluating e as a constant succeeds
}
```

This is probably useful, specifiable, and implementable. Would this be the right shape for this facility? One issue here might be that if `e` is expensive to evaluate, you probably don't want to first check if you can evaluate it and then, if you *can* evaluate it, actually evaluate it - this may require the compiler to actually evaluate it twice (not ideal) and definitely requires the user to type it twice (also not ideal).

The closest parallel to the desired semantics might be (as Nat Goodspeed pointed out) exceptions: you `try` to evaluate an expression, but allowing yourself to `constexpr catch` evaluation failures. That seems like the right semantic, and is actually a really interesting thing to consider. But that's… a large thing to

think about, with a lot of implications. It may be something we may want to pursue, and I'm sure sufficiently clever people can come up with interesting use-cases.

But for this problem, we have a fairly simple solution that involves no change in language semantics, simply exposing to the user something the compiler already knows. I think the tiny library facility is the right approach here. At least for now.

§ 3.2 Interesting Cases

Let's go over some interesting cases to flesh out how this facility should behave. With the older revisions of this paper, where the facility was specifically asking the question "is this an active member of a union?", these questions ended up being potentially subtle.

But with generalizing the facility to simply asking if an object is within its lifetime, they become more straightforward to answer.

§ 3.2.1 Nested Anonymous Unions

Consider:

```
union Outer {
  union {
    int x;
    unsigned int y;
  };
  float f;
};
consteval bool f(Outer *p) {
  return std::is_within_lifetime(&p->x);
}
```

If the active member of `p` is actually `f`, then not only is `p->x` not the active member of *its* union but the union that it's a member of isn't even itself an active member. What should happen in this case?

This should be valid and return `false`. It doesn't matter how many layers of inactivity there are - `p->x` isn't the active member of a union, and thus is not within its lifetime, and that's what we're asking about.

§ 3.2.2 Subobject

Consider:

```
struct S {
  union {
    struct {
      int i;
    } n;
  };
};

consteval bool f(S s) {
```

```
  return std::is_within_lifetime(&s.n.i);
}
```

Here, `s.n.i` is not a variant member of a union - it's a subobject of `s.n`. But that no longer matters - if `s.n` is within its lifetime, then `s.n.i` here would be too.

3.2.3 Other rules

There's a similar rule in this space, [expr.const]/5.8

(5.8) an lvalue-to-rvalue conversion unless it is applied to

(5.8.1) — a non-volatile glvalue that refers to an object that is usable in constant expressions, or

(5.8.2) — a non-volatile glvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of E;

With this revision, the facility exactly matches this bullet: "is this a pointer that I can read during constant evaluation?"

3.3 Alternative Naming

Should the name reflect that this facility can only be used during constant evaluation (`std::is_consteval_within_lifetime`) or not (`std::is_within_lifetime`)?

It would help to make the name clearer from the call site that it has limited usage. But it is already a `consteval` function, so it's not like you could misuse it.

3.4 Why a pointer rather than a reference?

This proposal is for asking `is_within_lifetime(&x)` rather than `is_within_lifetime(x)`. Why a pointer, rather than a reference?

There are a few arguments in favor of a pointer. First, we just don't have to worry about passing in a temporary. Second, many of the other low-level manipulation facilities also take pointers (like `construct_at`, `start_lifetime_as`, etc.). Third, there's this whole other set of questions about reference binding validity that come up ([CWG453]).

It's not like these are insurmountable difficulties, but the pointer API just doesn't have them, and isn't exactly either burdensome or inconsistent. So is it even worth dealing with them?

3.5 Implementation Experience

Daveed Vandevoorde has implemented R1 of this proposal in EDG. There's no real implementation burden difference between R1 and R2.

As pointed out by Johel Ernesto Guerrero Peña and Ed Catmur, this facility basically already works in gcc and clang, which has a builtin function (`__builtin_constant_p`) that be used to achieve the same behavior.

§ 4 Wording

Add to 21.3.3 [meta.type.synop]:

```
// all freestanding
namespace std {
  // ...

  // [meta.const.eval], constant evaluation context
  constexpr bool is_constant_evaluated() noexcept;
+ template<class T>
+   consteval bool is_within_lifetime(const T*) noexcept;
}
```

And add to 21.3.11 [meta.const.eval] (possibly this section should just change name, but it feels like these two should just go together):

```
constexpr bool is_constant_evaluated() noexcept;
```

1 *Effects*: Equivalent to:

```
if consteval {
  return true;
} else {
  return false;
}
```

2 [*Example 1*:

```
constexpr void f(unsigned char *p, int n) {
  if (std::is_constant_evaluated()) {           // should not be a constexpr
        if statement
    for (int k = 0; k<n; ++k) p[k] = 0;
  } else {
    memset(p, 0, n);                            // not a core constant
        expression
  }
}
```

— *end example*]

```
template<class T>
  consteval bool is_within_lifetime(const T* p) noexcept;
```

3 *Returns*: `true` if `p` is a pointer to an object that is within its lifetime ([basic.life]); otherwise, `false`.

4 *Remarks*: During the evaluation of an expression `E` as a core constant expression, a call to this function is ill-formed unless `p` points to an object that is usable in constant expressions or whose complete object's lifetime began within `E`.

5 [*Example 2*:

```
struct OptBool {
  union { bool b; char c; };

  // note: this assumes common implementation properties for bool and char:
  // * sizeof(bool) == sizeof(char), and
  // * the value representations for true and false are distinct
  //   from the value representation for 2
  constexpr OptBool() : c(2) { }
  constexpr OptBool(bool b) : b(b) { }

  constexpr auto has_value() const -> bool {
    if consteval {
      return std::is_within_lifetime(&b);   // during constant evaluation,
        cannot read from c
    } else {
      return c != 2;                        // during runtime, must read
        from c
    }
  }

  constexpr auto operator*() -> bool& {
    return b;
  }
};

constexpr OptBool disengaged;
constexpr OptBool engaged(true);
static_assert(!disengaged.has_value());
static_assert(engaged.has_value());
static_assert(*engaged);
```

— *end example*]

4.1 Feature-test Macro

Add a new feature-test macro to 17.3.2 [version.syn]:

```
#define __cpp_lib_within_lifetime 2023XXL // also in <type_traits>
```

5 References

[CWG453] Gennaro Prota. 2004-01-18. References may only bind to “valid” objects.
https://wg21.link/cwg453

[N4868] Richard Smith. 2020-10-18. Working Draft, Standard for Programming Language C++.
https://wg21.link/n4868

[P2641R1] Barry Revzin. 2022-10-11. Checking if a union alternative is active.
https://wg21.link/p2641r1

[P2641R2] Barry Revzin. 2023-02-07. Checking if a union alternative is active.
https://wg21.link/p2641r2

[P2641R3] Barry Revzin. 2023-05-16. Checking if a union alternative is active.
https://wg21.link/p2641r3

---

1. All references to this paper are to [N4868], C++20 for convenient and stable linking.↩